

Ineroperability Between Homogenous and Heterogenous Blockchains

1st Adam Svitek

Faculty of Informatics and Information Technology STU)

Slovak Technic University

Bratislava, Slovakia

xsviteka@stuba.sk

Abstract—This document is a model and instructions for $\text{\LaTeX}$. This and the `IEEEtran.cls` file define the components of your paper [title, text, heads, etc.]. ***CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

Blockchain interoperability is one of the essential requirements for modern decentralized systems. What was once mostly a matter of communication between separate blockchain networks has slowly evolved into more complex challenge of execution layers, heterogeneous architectures and different transfer mechanisms. Particularly, Ethereum ecosystem in current state extends beyond the main network (Layer 1) to a fast growing set of Layer 2 solutions that improve scalability and reduce transaction costs. As the number of users and decentralized applications who operates in these environments increases each day, seamless movement of assets between layers and ecosystems becomes an important concern.

Users nowadays may need to transfer assets from Layer 2 to Layer 1 and vice versa or even make transactions to another ecosystem such as Polkadot. Although there are many existing interoperability protocols and bridges already supporting these exchanges and transfers, they each solely focuses on one step of the transfer route. These mechanisms mostly differs in their interfaces, operations and transaction flow. As a result, multi-step transfers still remains split, technically demanding and prone to more errors.

This paper addresses the described transfer fragmentation problem by proposing a unified transfer flow connecting the Ethereum ecosystem with Polkadot ecosystem. The proposed solution combines two asset transfer mechanisms called Across from Ethereum asset transfers including tranfers within Layer 2 networks and Layer 2 to Layer 1 and vice versa , with Snowbridge as an interoperability bridge between Ethereum and Polkadot. This way, the solution enables a seamless transfer path from Ethereum L2 environments right to Polkadot through single web based aplication. Rather then forcing user to make more transactions across different

web applications, the proposed system aims to connect these interoperability multi steps into one process

II. BACKGROUND

A. Blockchain

Blockchain is a type of distributed ledger technology that enables a network of participants to maintain a shared and synchronized database without relying on a central authority [?], [?]. Transactions are grouped into blocks, which are crypto- graphically linked to form a chain; hence the name blockchain [?].

This structure ensures transparency and immutability, as altering any stored data would require modifying all subsequent blocks and gaining consensus from the majority of the network [?]. At the same time, blockchain addresses fundamental issues of digital systems such as double- spending, where the same asset could otherwise be used multiple times without proper verification.

B. Consensus, Proof of Work and Proof of Stake

Consensus mechanisms are a fundamental building block of blockchain systems, as they enable agreement on a single valid state of the network without centralized control. Among the most widely used mechanisms are Proof of Work (PoW) and Proof of Stake (PoS), both designed to ensure trust, security, and consistency in decentralized environments.

Proof of Work was first introduced in 2008 by Satoshi Nakamoto as part of Bitcoin. Its core idea is that creating a new block requires solving a computationally intensive problem. This process involves finding a nonce such that the resulting cryptographic hash satisfies predefined conditions, typically a certain number of leading zeros. This method, commonly referred to as mining, relies on trial and error and requires significant computational resources.

The security of Proof of Work is based on the high cost of computation. An attacker would need to control the majority of the network's computational power (known as a 51which would require substantial investment in hardware and energy. However, this approach has notable drawbacks, including high energy consumption, limited scalability, and a tendency toward centralization through mining pools.

To address these limitations, Proof of Stake was introduced as an alternative consensus mechanism. Instead of relying on

computational work, PoS is based on economic commitment. Participants, known as validators, are required to lock a certain amount of cryptocurrency as stake.

In this model, the probability of being selected to propose a new block depends on the size of the validator's stake. Once selected, the validator proposes a block that is verified by other participants in the network. Security is ensured through economic incentives, where dishonest behavior can lead to penalties such as slashing. Compared to Proof of Work, this approach significantly reduces energy consumption and improves scalability, while lowering hardware requirements. However, it introduces challenges such as potential centralization and increased protocol complexity.

C. Blockchain interoperability

Despite the security and reliability provided by consensus mechanisms, blockchain systems often operate as isolated environments, each defined by its own protocol, consensus rules, and execution model. This lack of standardization significantly limits their ability to communicate with one another, leading to interoperability challenges across different blockchain ecosystems [?], [?].

Blockchain interoperability refers to the ability of independent blockchain networks to exchange data and assets while maintaining security and correctness. As blockchain ecosystems evolve independently with different architectures and execution environments, enabling communication between them has become a key requirement for broader adoption of decentralized technologies [?], [?].

Interoperability solutions aim to bridge these isolated systems by enabling cross-chain communication. This includes asset transfers, message passing, and verification of state across different blockchains. Various approaches have been proposed, including notary schemes, relays, sidechains, and hash-locking mechanisms, each introducing trade-offs in terms of security, trust assumptions, and performance [?].

From a functional perspective, interoperability enables cross-chain asset transfers and supports decentralized applications operating across multiple blockchain networks. However, achieving such interoperability is non-trivial, as it requires coordination between heterogeneous systems that may differ in their trust models, consensus rules, and execution logic [?], [?].

Despite significant progress in research and development, many interoperability solutions remain limited to specific network pairs or use cases. As a result, users often need to interact with multiple systems, increasing operational complexity and introducing additional risks [?], [?].

D. Cross-chain vs Cross-layer

Cross chain and cross-layer represents two different approaches to communication within one and across another blockchain system with each addressing distinct challenges connected to interoperability and scalability.

Cross chain communication refers to interaction between two or more independent blockchain networks such as

Ethereum and Polkadot. Because of their independence they do not share the same consensus mechanism, state, or execution environment.

To partly solve this issue, various solutions were developed such as notary-based systems or lock-and-mint mechanisms. These approaches introduce trade-offs between trust assumptions, security, and complexity.

Cross layer is on the other hand a way of communication across the same network across multiple layers. The network is mostly structured into base layer referred to as Layer 1 and one or more secondary layers called Layer 2.

Unlike independent blockchains, Layer 2 systems inherit their security from the underlying Layer 1, simplifying interoperability but still introducing complexity in asset movement.

E. Ethereum

Ethereum is decentralized blockchain platform that implements the functionality of smart contracts and their executions [?], [?]. Smart contracts are self executing programs stored on the blockchain that automatically enforce predefined rules without any need of centralized controlling force [?].

Ethereum operates as a Layer 1 blockchain, providing consensus, security and data availability. To improve scalability, Layer 2 solutions were introduced, which execute transactions off-chain while relying on Layer 1 for security.

This layered system introduces cross-layer interactions, where assets are transferred between Layer 1 and Layer 2, increasing the need for interoperability solutions.

F. Snowbridge

Snowbridge is an interoperability protocol designed to communicate between Ethereum and Polkadot ecosystem while minimalizing trust between them.

Snowbridge uses light clients, which allows one blockchain to verify the state of another blockchain without relying on any external trust. Specifically, the protocol enables Polkadot to check Ethereum block headers and events and other way around, thus ensuring the cross-chain messages are validated based on cryptographic proofs rather than trusted middle forces. This system improves security compared to other bridges which uses centralized validators.

Shortly said, Snowbridge enables the transfer of assets and messages between Ethereum and Polkadot Parachains. For example, assets that are locked on Ethereum can be used and transferred with their representation on Polkadot and also messages and assets with Polkadot origin can be used on Ethereum.

This approach also comes with its downsides such as higher complexity meaning higher computational costs compared to simpler bridging solutions.

G. Across protocol

Across protocol is an interoperability solution designed for more efficient asset transfers across Ethereum network system, mostly between Layer 2 networks and also Layer 1 Ethereum.

Unlike other bridges that rely mostly on locking assets and minting their representation on another chain, Across implemented an intent-based architecture combined with relayer-driven liquidity model. The main goal of this solution is to drastically reduce transfer latency while maintaining security provided by Ethereum.

In reality, the user starts the transfer by filling the required information for transfer(intent) such as origin chain, destination chain, assets recipient. After depositing assets on the origin chain, relayers provide the liquidity on the destination chain by sending funds directly to designated recipient. This enables almost instant transfer speed without waiting for cross chain finality.

Across uses an *optimistic security model* where transactions created are assumed valid but can be challenged during some period of time. The key feature is separation between fast execution and delayed settlement. The users receive assets almost instantly compared to other systems; the relayers have to wait longer for transfer finalization.

III. RELATED WORKS

Existing interoperability solutions aim to connect fragmented blockchain ecosystems by enabling asset transfers and messaging across different networks. These proposed and used solutions differ in their working architecture, trust assumptions and performance. Where some protocols aim to connect layers inside one system, others try to achieve better cross chain interoperability.

Interoperability protocols can be divided into 2 main groups, those that want to achieve faster transfer speed by introducing liquidity pools or relayers where the main downside is additional trust assumptions or delayed settlement with relayers and others focusing mostly on minimizing trust to reduce reliance on central entity by using cryptographic verification mechanisms such as light-clients or zero-knowledge proofs.

While these solutions improve usability and performance, most of them are limited and focused on interoperability among one system or compatible blockchain environments creating a need for a combined and seamless solution to transfer assets.

A. HyperBridge

Hyperbridge is a interoperability protocol designed to enable secure communication between heterogeneous blockchain systems using cryptographic verification. Hyperbridge is based on verifiable state and consensus proofs, allowing one chain to verify the state of the other independently.

The main advantage of Hyperbridge is in its strong trust-minimized security model. By relying on cryptographic proofs instead of external validators or liquidity providers, it significantly reduces trust assumptions and improves the overall integrity of cross-chain communication. Also, its design allows interoperability between different blockchains instead of allowing transfers in only one single ecosystem.

However this design comes with increased complexity because of the generation and verification of cryptographic

proofs. With increased complexity comes higher computational cost and therefore increased transaction fees, and higher latency.

B. TurtleV2

Turtle was an interoperability platform developed by Velocity Labs that aimed to simplify cross-chain asset transfers within the Polkadot Ecosystem. Turtle did not introduce a new interoperability mechanism, but instead integrated multiple existing protocols into unified workflow. Its goal was to ease the complexity of multi-step cross-chain operations and combine them into single step action.

The platform focused on scenarios involving transfers across multiple layers and ecosystems such as bridging assets from Ethereum to Polkadot and then routing them to specific parachains. To achieve their goal, Turtle combined Snowbridge protocol for Ethereum-Polkadot transfers and XCM for internal Polkadot transfers. This enabled seamless workflow from Ethereum to Polkadot and its parachains within one single user interface. Because of using other protocols, Turtle did not improve the security and trust.

C. Hop Protocol

Hop protocol is an interoperability solution designed to bring fast and efficient transfers between Ethereum L1 and L2 and also L2 and L2 networks. The main goal is to reduce the delays connected with rollup withdrawals and improve efficiency of cross-layer transfers inside Ethereum ecosystem. The protocol allows users to move assets across networks with much lower latency compared to native bridging mechanisms, and improving overall usability. [?]

The main advantage of Hop Protocol is the ability of nearly instant transfers across Layer 2 networks, creating a smooth experience for users that need to transfer assets more frequently between rollups without waiting for settlement periods. The main downside is the need of sufficient liquidity to function efficiently. [?]

D. cBridge

Celer cBridge is an interoperability solution designed to enable faster asset transfer across more blockchain networks including also Ethereum L1 and L2 ecosystems. As other protocol with its aim to increase speed of transfer, cBridge allows users to transfer assets without the need of waiting for finality. On the other hand as other cBridge also relies on liquidity providers and validators, increasing trust assumptions compared to fully trust-minimized solutions. [?]

IV. PROPOSED SOLUTION

The proposed solution addresses the complexity of asset transfers across homogenous and heterogeneous environments, mostly between Ethereum based networks and the Polkadot ecosystem. Today's interoperability solutions often require users to interact with more than one interface what makes it prone to user based errors during the transfer. To reduce this complexity in multi transfers, the proposed solution introduces

a unified application layer which combines two protocols and combining them into one single user-oriented interface.

The main part of the solution is to simplify the transfer while keeping the user in charge setting the essential parameters like, source chain, destination chain, amount, token, sender wallet and recipient address while the system determines and executes the required series of operations.

Rather than creating a new transfer protocol, the system takes the use of two previously mentioned protocols being Across protocol and Snowbridge, and combines them into one single transfer workflow.

A. High-Level Design

The proposed system is designed as a unified application layer built on top of existing interoperability protocols. Its purpose is to simplify asset transfers between Ethereum-based networks, including both Layer 1 and Layer 2, and the Polkadot ecosystem.

Instead of introducing a new transfer mechanism, the system integrates existing protocols into a single workflow. This approach allows the application to abstract the complexity of cross-chain and cross-layer operations while preserving the underlying security and execution models of the used protocols.

At a high level, the system separates user interaction from protocol execution. The user defines only the essential transfer parameters, while the application is responsible for constructing and executing the required sequence of operations. This separation enables a clear abstraction between the input provided by the user and the underlying transfer logic.

The design follows a modular and extensible approach, where different interoperability mechanisms can be combined within a single transfer process. This makes it possible to support both simple and complex transfer scenarios without requiring additional interaction from the user.

By acting as an orchestration layer above existing protocols, the system reduces the need for users to manually coordinate multiple steps across different platforms, improving usability and reducing the likelihood of errors.

B. System Overview

The system is implemented as a client-side application that communicates directly with external interoperability services and blockchain networks. Instead of relying on a dedicated backend, the application acts as an orchestration layer that connects user input with protocol execution.

The architecture consists of several logical components. The catalog layer is responsible for maintaining a unified representation of supported networks and assets. For Ethereum-based environments, chain and token data are obtained dynamically from the Across API. For the Polkadot ecosystem, Asset Hub networks are integrated into the same representation, allowing both ecosystems to be handled in a consistent way.

A central component of the system is the transfer planner, which constructs a transfer plan based on user-defined parameters. The output of the planner is a structured sequence of

steps that defines how the transfer will be executed across different environments.

The execution layer is responsible for processing the generated transfer plan. It consists of protocol-specific modules that prepare and submit transactions to the corresponding networks. One module handles transfers within Ethereum-based networks using Across Protocol, while another module is responsible for interoperability between Ethereum and Polkadot using Snowbridge.

The user interface serves as the main interaction point and reflects the internal state of the system. It collects transfer parameters, presents the generated transfer plan, and provides feedback during execution. This ensures that the user remains informed about the progress and outcome of the transfer without interacting directly with underlying protocols.

C. Transfer Flow

The transfer flow starts in the user interface. The user first selects the source network. Based on this choice, the application loads the tokens that are available on the selected network. After the token is selected, the system loads the possible destination routes.

Next, the user enters the transfer amount and the recipient address. The recipient address is checked to make sure it has the correct format. The address format also changes based on the destination address. If the destination chain is in Polkadot ecosystem, the address is different from the one in Ethereum network. When all required fields are filled, the user clicks the preview button to reveal the transfer plan prepared by application. This plan shows a simple overview of the steps that will follow, what enables user to review the plan before starting actual execution of the transfer.

Once the transfer is confirmed by selecting submit button, the application starts the execution plan. In transfer flow, each step is processed in order. For every step, the required transaction is prepared and sent to the connected wallet for its approval by the user, and after the user confirms it, the transaction is submitted.

If the transfer contains multiple steps, the application waits for first step to finish before moving to the other to mitigate the errors of low balance if user doesn't own any tokens on the middle chain (Layer 1 Ethereum chain). In this case, the output from the previous step is used in the following step.

During the whole process, the application shows the current progress of the transfer. The user can see which steps were completed, which step is currently running, and whether an error occurred. After all steps are completed, the final transfer result is shown.

V. IMPLEMENTATION

The proposed solution is implemented as a client-side web application built with React, TypeScript, and Vite. The application communicates directly with external APIs, EVM RPC endpoints, and the connected wallet, namely MetaMask. This architecture keeps the system lightweight, while the frontend application itself coordinates the complete transfer flow.

A key implementation element is the internal data model used to represent transfer inputs and generated execution steps. The user input is first transformed into a structured transfer intent containing the selected environment, source chain, destination chain, asset, amount, and recipient. This intent is then processed by the planner, which converts it into an internal transfer plan. The plan does not correspond directly to a single blockchain transaction, but rather to a sequence of protocol-level execution steps. In practice, one planned step may still require multiple on-chain transactions during execution, for example in the Snowbridge flow.

The application uses a unified internal model for supported chains and transferable assets in the user interface. For Ethereum-based transfers, available chains, supported tokens, and destination routes are derived dynamically from the Across API route response. This means that the frontend does not hardcode supported EVM routes, but obtains them from the protocol response, which makes future extensions easier if Across expands its supported chains, assets, or routes. On the Polkadot side, Asset Hub destinations for either testnet or mainnet are integrated into the same internal chain model, so Ethereum and Polkadot environments can be handled by a single planning flow. In practice, this allows the interface to work with one consistent representation of transfer targets, even though the underlying systems use different addressing and execution models.

The central implementation logic is located in the planner. Its task is to map a transfer request specified by the user into an executable path. If the destination is a supported EVM chain, the planner generates a single-step transfer using Across Protocol. If the destination is Asset Hub and the source is Ethereum Layer 1, the planner generates a single Snowbridge step. If the destination is Asset Hub and the source is a supported Layer 2 network, the planner creates a two-step plan consisting of an Across transfer from Layer 2 to Layer 1, followed by a Snowbridge transfer from Layer 1 to Asset Hub. In the current implementation, Snowbridge routes are available only for native ETH liquidity on supported source chains.

The Across integration is implemented as a dedicated execution module for EVM-based route transfers. It is responsible for switching the wallet to the correct source network, preparing transfer amounts in the expected on-chain format, requesting approval when required, and submitting the final bridge transaction. In combined transfers, the application also monitors the status of the submitted Across deposit and waits until the transfer is filled before continuing with the next step. This is necessary because the Snowbridge step can begin only after the Layer 2 to Layer 1 transfer has been completed.

An important technical detail is asset representation. Although the user selects ETH liquidity for Snowbridge-compatible routes, the Ethereum side of the Snowbridge transfer operates with WETH. Because of this, the implementation includes explicit asset handling logic that wraps ETH to WETH on Layer 1 before the Snowbridge transfer is submitted. This ensures compatibility between user input,

the planned route, and the asset that is actually transferred to Asset Hub.

Another important implementation aspect is sequential execution. After the transfer plan is generated and confirmed by the user, the system processes the plan one step at a time. Each step produces either a final result or an intermediate output that is required by the next step. This is especially important in the case of combined transfers, where the Snowbridge step reuses the destination-side output amount determined for the Across step instead of treating the second transfer as an isolated action. The execution layer therefore does not handle individual transfers as separate operations, but as dependent parts of one coordinated process.

The application also maintains an explicit execution state for user feedback. During processing, the interface tracks progress for individual actions such as approval, bridge submission, waiting for Across fill, WETH preparation, and Snowbridge approval. Each tracked action moves through simple states such as idle, running, success, or error. This state tracking is not only a user interface feature, but also part of the coordination logic, since the next action depends on the successful completion of the previous one. In this way, the implementation combines protocol-specific execution with application-level coordination and monitoring, which is necessary for reliable multi-step interoperability workflows.